

Great Expectations

Data quality, made declarative — starting with GX Core 1.0

```
expect(your_data).to_be(trustworthy)
```

Bad data is expensive

Every dashboard, report, and ML model is only as good as the data underneath it. When bad data slips through, the damage shows up far downstream — where it's hard to trace and costly to fix.

The 1×10×100 rule

\$1 to prevent a data error at entry

\$10 to fix it later in the pipeline

\$100 once a bad decision is already made

Catching data problems early is the cheapest quality you can buy.

1

Wrong decisions

Leaders act on numbers that were never correct.

2

Broken dashboards

A schema change silently empties a chart.

3

Failed ML models

Garbage in, garbage out — models learn the errors.

4

Lost trust

Once burned, stakeholders stop believing the data.

What do we mean by "data quality"?

Quality isn't one thing — it's a handful of properties you can actually check. GX lets you assert each of these.



Validity

Values follow the rules — types, ranges, formats.



Completeness

Nothing important is missing — no unexpected nulls.



Accuracy

Values reflect the real world correctly.



Consistency

Data agrees with itself across tables & time.



Uniqueness

No duplicate rows or keys where there shouldn't be.



Timeliness

Data is fresh and arrives when expected.

THE BIG IDEA

You already unit-test your **code**. Why not unit-test your **data**?

Tests for code catch broken logic before it ships. Great Expectations gives you the same safety net for the data flowing through your pipelines — an automatic check that says "this is what good data looks like," every single run.

What is Great Expectations?

The most popular open-source data quality framework in the world — a Python library that helps data teams manage their complete data-quality lifecycle.



Test

Validate data you produce or ingest from other teams.



Alert

Get notified the moment an expectation fails.



Collaborate

Share definitions of “good data” with every stakeholder.



Document

Auto-generate living docs of your data's expectations.

At its core:

Expectations

Declarative statements that make verifiable assertions about your data.

“Unit tests for your data.”

Open source

Apache 2.0, community-driven, free to run anywhere.

Runs on Python

One pip install and an import to get going.

Expectations: assertions about your data

An Expectation reads almost like plain English. You pick the ones that fit your data, validate them, and review the results — that's the whole loop.

In plain English

"The passenger count should never be more than 4 — a NYC taxi only holds 4 people."

```
gx.expectations.ExpectColumnMax\
  ToBeBetween(\
    column="passenger_count",
    min_value=0,
    max_value=4,
  )
```

1 Choose

Pick expectations that fit your data

2 Validate

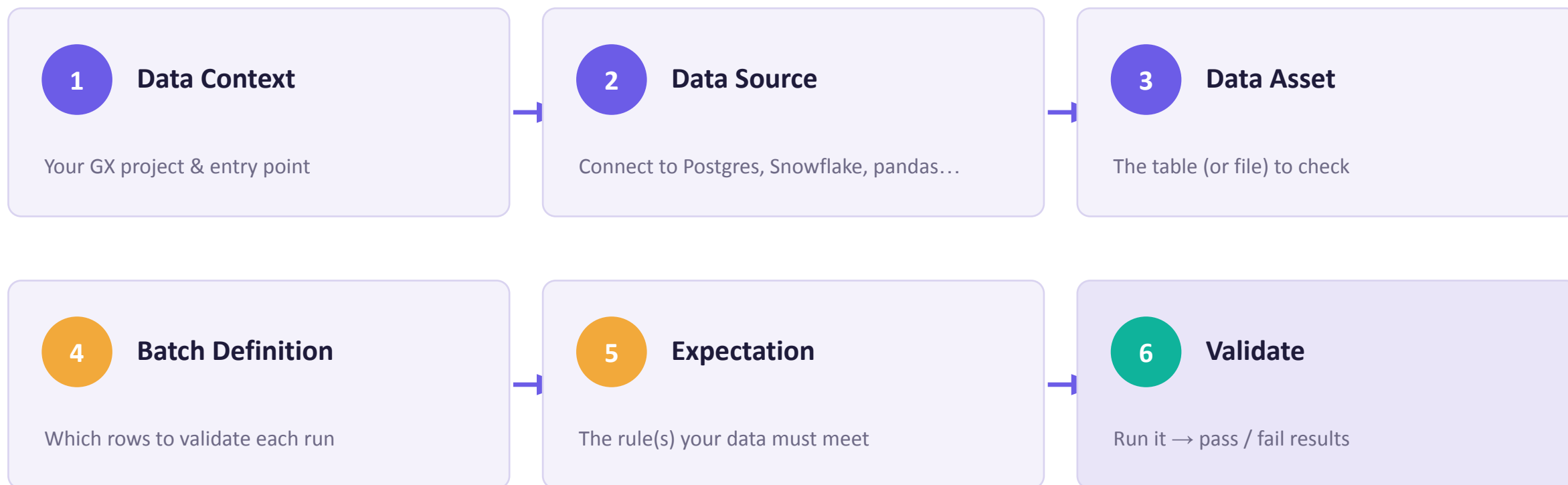
Run them against a batch of data

3 Review

Read pass / fail results

The GX Core workflow

Six small steps take you from “no GX” to “validated data.” You set up the first four once, then reuse them every run.



set up once → reuse every run

Key GX terms, in one line each

Data Context

The entry point that manages all your GX config.

Data Source

A connection to where your data lives.

Data Asset

A specific table or file within a source.

Batch Definition

Which slice of the asset to validate.

Expectation

A single verifiable assertion about the data.

Expectation Suite

A named bundle of Expectations.

Validation Definition

A Suite paired with the data to run it on.

Checkpoint

Runs validations and triggers actions/alerts.

Install GX & create a Data Context

GX runs on Python. Install it once, import it, and create a Data Context — the single entry point to everything GX does.

- The Context manages your configuration and components.
- It can be ephemeral (in-memory) or file-backed (saved to disk).
- However you store it, the API behaves the same.

Think of the Context as “your GX project.”

```
# 1. Install once, in your venv or VM
pip install great_expectations

# 2. Import GX
import great_expectations as gx

# 3. Create the Data Context
context = gx.get_context()
#    -> file-backed or ephemeral
```

Connect to a Data Source & Asset

Tell GX where your data lives (the Data Source), then point at the specific table you want to check (the Data Asset).

- One Source can expose many Assets (tables).
- The Asset is what you'll write Expectations about.
- Postgres shown here — many other backends supported.

Supported: pandas, Spark, PostgreSQL, Snowflake, Databricks, BigQuery & more.

```
# Connect to a data source (Postgres)
ds = context.data_sources.add_postgres(
    "my_postgres",
    connection_string=CONNECTION_STRING,
)

# Point at a table -> a Data Asset
asset = ds.add_table_asset(
    name="taxi_data",
    table_name="nyc_taxi_data",
)
```

Define the Batch to validate

A Batch Definition describes exactly which data gets validated in each run — the whole table, or a subset like a single day or month.

- Whole-table is the simplest starting point.
- Subsets let you validate incrementally (e.g. daily loads).
- Same definition is reused on every pipeline run.

One Asset can have several Batch Definitions for different slices.

```
# What data to validate in each run
batch_def = asset.\
    add_batch_definition_whole_table(
        "all_rows"
    )

# (Or a subset: by day, month,
#  a custom query, and so on.)
```

Write your first Expectation

Now the fun part: assert something about the data. GX ships a large library of ready-made Expectations for common checks.

- Here: the max passenger_count should be between 0 and 4.
- NYC taxis hold up to 4 people... or do they?
- Hundreds of built-in Expectations cover most needs.

Expectations read like plain English — that's the point.

```
# An Expectation = a verifiable assertion
expectation = gx.expectations.\
    ExpectColumnMaxToBeBetween(
        column="passenger_count",
        min_value=0,
        max_value=4,    # taxis hold 4... right?
    )
```

Validate — and read the result

Get a batch, run the Expectation against it, and review the outcome. GX tells you exactly what passed, what failed, and why.

```
batch = batch_def.get_batch()
result = batch.validate(expectation)

print(result.success)
```

FAILED`result.success` → **False**

Some rides actually have 6 passengers — turns out NYC has minivan taxis that seat 6! Our rule was wrong, not the data.

The fix: update the Expectation and re-run.

```
expectation.max_value = 6
result = batch.validate(expectation)
print(result.success)
```

PASSED`result.success` → **True**

A few lines of code found a real gap in our understanding of the data — that's data quality at work.

PUTTING IT ALL TOGETHER

The whole GX Core flow — about a dozen lines

```
import great_expectations as gx

context = gx.get_context()
ds       = context.data_sources.add_postgres("db", CONN)
asset    = ds.add_table_asset("taxi", "nyc_taxi_data")
batch_def = asset.add_batch_definition_whole_table("all")

exp = gx.expectations.ExpectColumnMaxToBeBetween(
    column="passenger_count", min_value=0, max_value=6)

batch  = batch_def.get_batch()
result = batch.validate(exp)
print(result.success)           # -> True
```

Connect context → source → asset

Scope batch definition

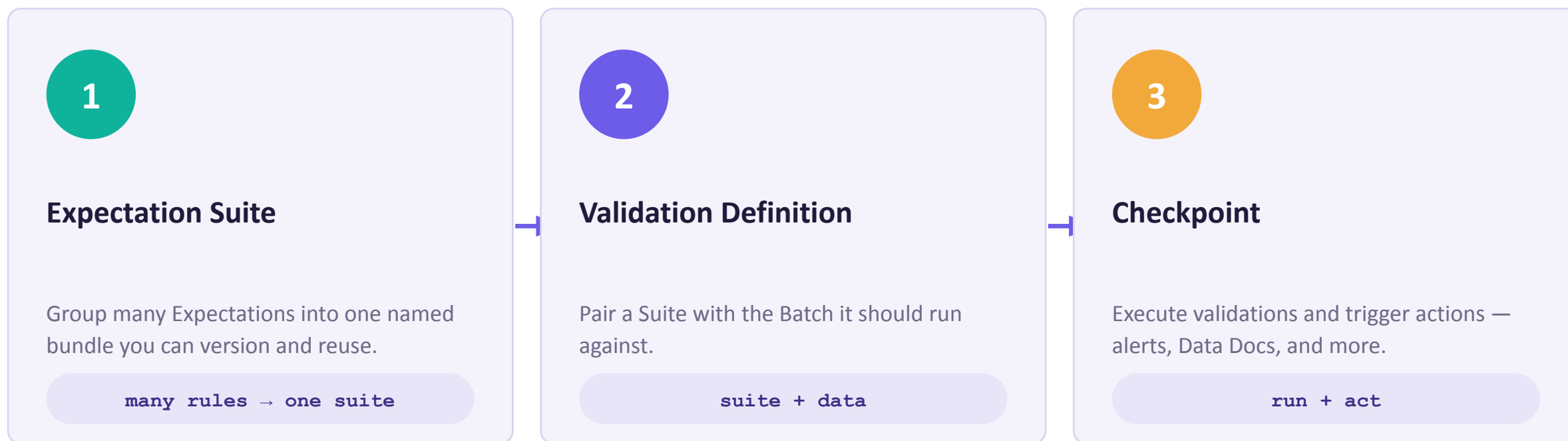
Assert an Expectation

Validate get pass / fail

That's the entire loop you just learned.

Scaling up: Suites, Validations & Checkpoints

One Expectation is great for a demo. Real pipelines bundle many checks and run them automatically, on a schedule, with alerts.



This is how GX moves from a notebook experiment to an automated quality gate in your data pipeline.

Data Docs: your data's living documentation

Every validation can render Data Docs — clean, human-readable pages that show what you expect of your data and how each run performed.



A single source of truth

Everyone sees the same definition of “good data.”



Faster onboarding

New teammates learn a dataset by reading its expectations.



Trust with stakeholders

Show, don't tell: proof that data was checked.



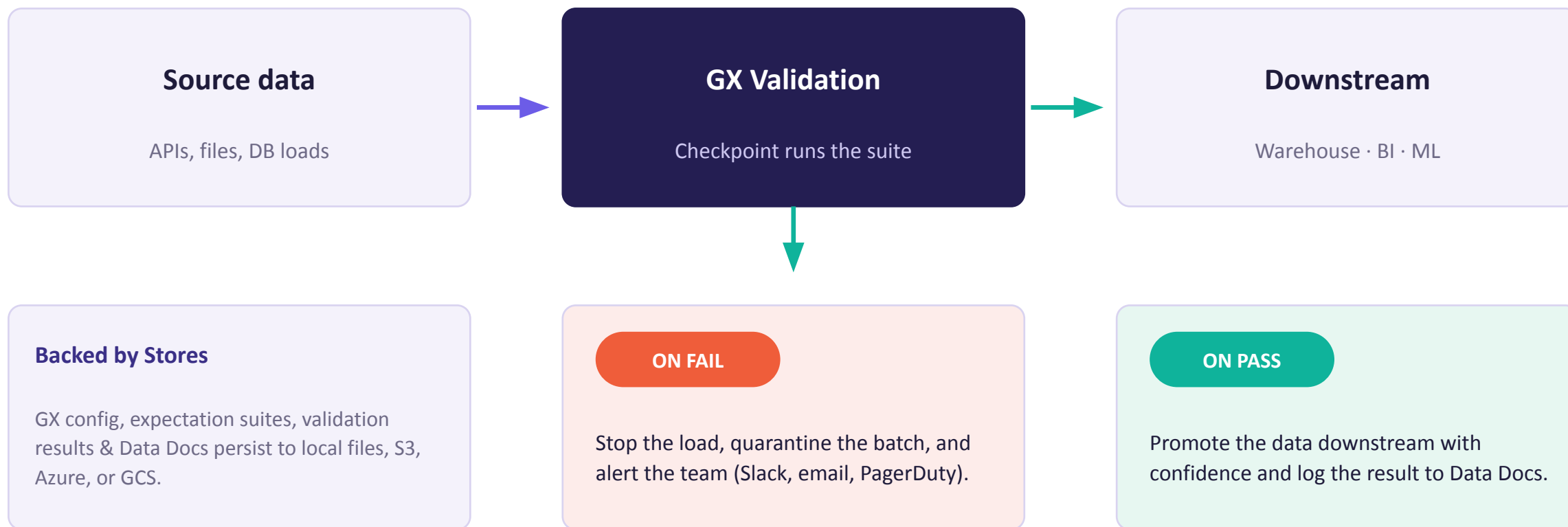
Captured tribal knowledge

Assumptions live in code & docs, not in someone's head.

Data Docs · nyc_taxi_data	
Expectation	Status
column max passenger_count ≤ 6	PASS
passenger_count not null	PASS
fare_amount ≥ 0	PASS
pickup < dropoff time	FAIL
Illustrative — GX renders richer HTML.	

Where GX sits in your data pipeline

GX acts as a quality gate between raw data and everything that depends on it. Good data flows on; bad data is caught, quarantined, and alerted on.



GX Core vs GX Cloud

Start free and local with GX Core. Reach for GX Cloud when you want a managed, collaborative platform on top.

GX Core

open source · the focus of today

- Free, Apache-2.0, self-hosted
- Python library + CLI workflow
- Full control in your own environment
- Define, validate & build Data Docs
- Perfect for learning & pipelines-as-code

GX Cloud

managed platform

- Hosted web UI, no infra to run
- Scheduling & automated runs
- Built-in alerting & notifications
- Collaboration for non-coders too
- Managed storage & dashboards

PART 2

Profiling vs Validation

Where does ydata-profiling fit — and when should you use it instead of (or alongside) GX?

What is ydata-profiling?

One line of code that generates a rich, interactive report about a dataset — the fastest way to understand data you've never seen before. (Formerly pandas-profiling.)

Distributions & stats

Min, max, mean, quantiles per column

Missing values

Where and how much data is absent

Correlations

How columns relate to each other

Cardinality & duplicates

Unique counts and repeated rows

Alerts / warnings

Auto-flags of skew, zeros, high missingness

Pandas & Spark

Works on DataFrames big and small

```
from ydata_profiling import \
    ProfileReport

ProfileReport(df).to_file("eda.html")
```

It answers one question:

“What's in my data?”

It describes and explores — it does not pass or fail, and it doesn't run as a gate in your pipeline. Think discovery, not enforcement.

ydata-profiling vs GX: when to use which

	ydata-profiling	Great Expectations
Purpose	Explore & understand data	Validate & enforce data quality
Answers	“What's in my data?”	“Is my data still correct?”
Output	One-off HTML report + stats	Pass / fail results + Data Docs
When you run it	Ad-hoc, at the start of a project	Every run, in CI/CD & pipelines
Pass / fail?	No — descriptive only	Yes — assertion-based
Alerting	No	Yes — on failure
Best for	Discovery & first-look EDA	Monitoring & regression safety

They're not rivals — profiling helps you discover the rules; GX enforces them.

Better together: discover, then enforce

The two tools shine in sequence. Profile a new dataset to learn its shape and spot candidate rules, then codify those rules as GX Expectations that run forever.



Heads-up: ydata-profiling once auto-exported a GX suite via `to_expectation_suite()`, but that built-in bridge is deprecated in current versions — translate the insights into Expectations yourself.

Key takeaways

1

Expectations are unit tests for data

Declarative, verifiable, readable assertions.

2

GX Core is free & code-first

context → source → asset → batch → expectation → validate.

3

Scale with Suites & Checkpoints

From one check to an automated quality gate with alerts.

4

Profile to discover, GX to enforce

ydata-profiling explains data; GX protects it.

Next steps

docs.greatexpectations.io

- “Try GX Core” quickstart
- Browse the Expectations gallery
- Explore GX Cloud

```
pip install great_expectations
```

Questions? Let's validate some data.